Application of Curiosity Driven Exploration Algorithms for Hardware Interference Identification

12 octobre 2026

Ludovic Matar | 12 octobre 2026



Inria – Centre Inria de l'Université de Bordeaux

Content

1. Context

2. Our simplified simulator

3. Exploration method

4. Results

5. Conclusion

Outline

1. Context

2. Our simplified simulator

3. Exploration method

4. Results

5. Conclusion

Context

- The shift from single-core to multi-core architectures is essential in safety-critical embedded systems in multiple domains.
- However, this transition introduces new complexities due to tasks running in parallel competing for shared resources ; inter-core interference.
- The AMC 20-193 standard requires identifying all sources of interference that can occur when executing tasks in parallel and addressing them.

Purpose

The purpose of the study is to demonstrate the ability of curiosity-driven exploration algorithms to be used as a tool for exhibiting complex intercore interference phenomena using a simulated architecture. We justify that collected interference events can later be analyzed by experts to identidy their sources.

- We show that one can obtain a great diversity of interference features by using available detailled information from the simulated platform.
- We demonstrate that with limited experimental budget, our proposed method discovers more behaviors than sampling entries of the simulator randomly.
- We provide guidance for analyzing the results.

Interference sources

Definition

We call a "pair of programs" a couple of two programs of code S_1 and S_2 allocated respectively to the C_1 and C_2 cores of an execution platform. S_1 and S_2 are independent code programs. In particular, they do not exchange data.

One can distinguish distinct categories of interference occurring because of contention on shared resources :

- **Cache contention :** When multiple cores compete for space in shared caches, evicting each other's data and causing cache misses.
- **Memory contention :** When multiple cores access a shared memory simultaneously (such as the main memory), leading to a bottleneck due to limited memory bandwidth.
- **Bus contention :** When cores compete for access to a shared bus, resulting in bandwidth limitations and delays in data transfer.
- **I/O Contention :** When multiple processes try to perform I/O operations at the same time, leading to delays and reduced I/O performance.

Outline

1. Context

2. Our simplified simulator

3. Exploration method

4. Results

5. Conclusion

Our simplified simulator

Our simulator model contains the following **resources** :

- two cores,
- their respective private L1 caches,
- a shared L2 cache unit
- DDR
- DDR controller,
- an interconnect model.

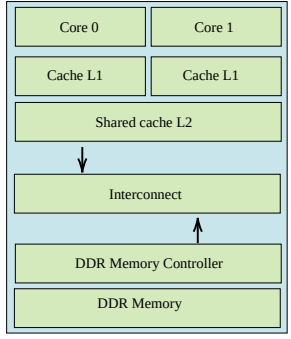


Figure – Simulated dual-core architecture used in this work.

How the simulator works

- Takes simplified assembly memory instructions
- One models distinct sets of addresses for the cores, i.e., core 0 : 0,9 and core 1 : 10,19.

```
1 program_core_0 = {23:('read', 11),37:('read',17),49:('write',6)}
2 program_core_1 = {12:('write', 24),18:('read',39),32:('write',37)}
```

- Outputs description of interference events for all resources.

Resources	Description	Features
L2 cache interference	Cross-core eviction tracking	set_idx, evicted_core_id, evicted_instr_id, evicted_addr, causing_core_id, causing_addr
BUS contention	Delayed requests due to bus bandwidth limitations	nb_competing_requests
DDR interference	Delayed requests due to bank conflicts	row,bank,addr,req_type,scheduled_delay,c ore_id
DDR scheduler interference	Delayed requests due to higher priority requests	bank,addr,req_type,scheduled_delay,core_id, core_attacker,type_attacker, addr_attacker,bank_attacker,row_attacker

Table – Events tracked by the simulator

Outline

1. Context

2. Our simplified simulator

3. Exploration method

4. Results

5. Conclusion

Overview of the method

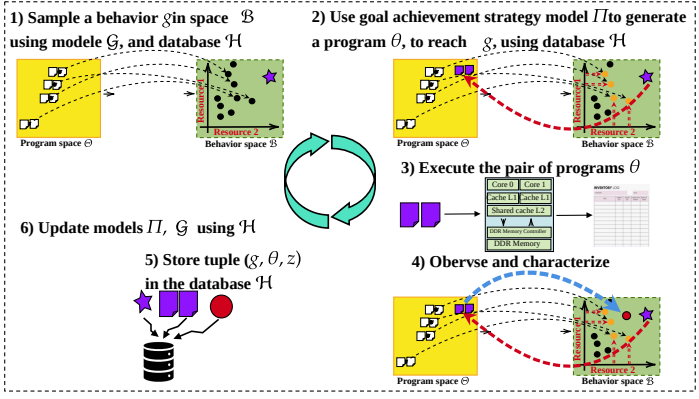


Figure – Schematic view of an IMGEP for our case of interference identification, based on the general description of IMGEP

Behavior space and goal sampling

- We aim at covering the behavior space $\mathcal{B}$ of interference on the architecture.
- The behavior space $\mathcal{B}$ thus made from several parts $\mathcal{B}_r$ for each resource $r : \mathcal{B} = \cup_r^n \mathcal{B}_r$. For every experiment on the simulator, events for resources r are stored in part $\mathcal{H}[r]$ of $\mathcal{H}$.
- One wants to discover as many interference as possible among all the possible combinations of resources,
- One chooses to bias the discovery process by sampling goals.

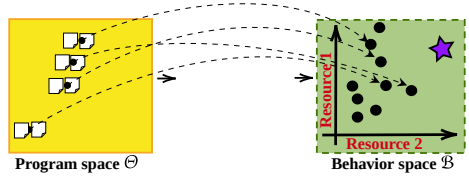


Figure – Schematic view of the goal sampling process. The purple star represents the goal in the behavior space. The axis indicate the agent tries to reach an experiment outcome with at least one interference for each required resource.

Two levels for the goal sampling

First level of goal sampling. The objective is to found a pair of program that exhibits the chosen goal a^*.

- The powerset $\mathcal{P}(\mathcal{C})$ of the resource set $\mathcal{C}$ ={**DDR, DDR scheduler, Cache, Bus**} contains $2^4 - 1 = 15$.
- Example :A experience outcome qualified by a_4 means that at lest an interference on the L2 cache was found, but none for the other resources.
- The random variable g of support $\mathcal{S} = \cup_j^{card(\mathcal{P}(\mathcal{C}))}\{a_j^*\}$ determines what goal the agent chooses.
- Needs to find an optimal law of g $\forall a^* \in \mathcal{S}, \mathbb{P}_{\mathcal{H},g}(\{a^*\}) = \mathbb{P}_{\mathcal{H}}(g = a^*) \in [0, 1]$

Reference	resource combinations
a_1	**BUS**
a_2	**DDR**
a_3	**DDR scheduler**
a_4	**L2 cache interference**
a_5	**DDR, BUS**
a_6	**DDR scheduler BUS**
a_7	**L2 cache, BUS**
a_8	**DDR, DDR scheduler**
a_9	**L2 cache, ddr**
a_{10}	**L2 cache, DDR scheduler**
a_{11}	**DDR, DDR scheduler, BUS**
a_{12}	**L2 cache, DDR, BUS**
a_{13}	**L2 cache, DDR scheduler, BUS**
a_{14}	**L2 cache, DDR, DDR scheduler**
a_{15}	**L2 cache, DDR, DDR scheduler, BUS**

Table – resource combinations

Two levels for the goal sampling

First level of goal sampling. The objective is to found a pair of program that exhibits the chosen goal a^*.

- Uniform sampling over $\mathcal{S}$:
  $\mathbb{P}_{\mathcal{H},g}(\{a^*\}) = \frac{1}{card(\mathcal{S})}, \forall a^* \in \mathcal{S}$
- Sampling based on exploration difficulty :
  $\mathbb{P}_{\mathcal{H},g}(\{a^*\}) = \frac{exp(-n_{\mathcal{H}}(a))}{\sum_j exp(-n_{\mathcal{H}}(a_j))}, \forall a^* \in \mathcal{S}$
- Diversity progress based sampling :$\mathbb{P}_{\mathcal{H},g}(\{a^*\}) = \frac{n_{\mathcal{H}}(a)}{\sum_j n_{\mathcal{H}}(a_j)}, \forall a^* \in \mathcal{S}$

Reference	resource combinations
a_1	**BUS**
a_2	**DDR**
a_3	**DDR scheduler**
a_4	**L2 cache interference**
a_5	**DDR, BUS**
a_6	**DDR scheduler BUS**
a_7	**L2 cache, BUS**
a_8	**DDR, DDR scheduler**
a_9	**L2 cache, ddr**
a_{10}	**L2 cache, DDR scheduler**
a_{11}	**DDR, DDR scheduler, BUS**
a_{12}	**L2 cache, DDR, BUS**
a_{13}	**L2 cache, DDR scheduler, BUS**
a_{14}	**L2 cache, DDR, DDR scheduler**
a_{15}	**L2 cache, DDR, DDR scheduler, BUS**

Table – resource combinations

Two levels for the goal sampling

Second goal sampling level.

- Firstly, the agent samples a^* using the law $\mathbb{P}_{\mathcal{H},g}$ of g.
- Secondly, it samples vectors g_r associated to the resources or distinct type events $r \in a$.

$$g_r \sim \mathcal{U}([f_1 \cdot (min_{\mathcal{B}_r} z)_1, f_2 \cdot (min_{\mathcal{B}_r} z)_2]) \otimes \cdots \otimes \mathcal{U}([f_1 \cdot (min_{\mathcal{B}_r} z)_{\dim(\mathcal{B}_r)}, f_2 \cdot (max_{\mathcal{B}_r} z)_{\dim(\mathcal{B}_r)}])$$

Goal achievement strategy

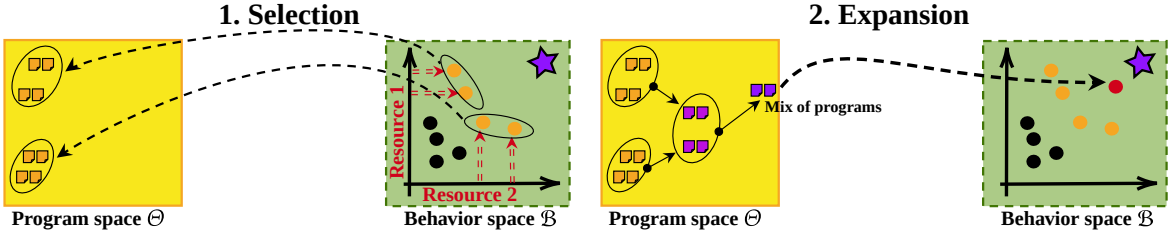


Figure – Schematic view of the goal achievement strategy for a combination of two resources. For each resource $r \in c$ the combination, one selects the k closest outcomes from the database $\mathcal{H}$ (yellow), and one expands by producing a candidate parameter (purple), and this involves first mixing the selected pairs of programs for resource 1 and resource 2 seperatly, then mixing the results.

Outline

1. Context

2. Our simplified simulator

3. Exploration method

4. Results

5. Conclusion

Results

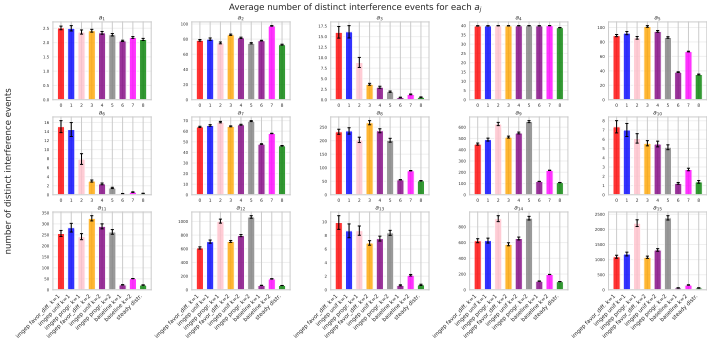


Figure – Number of distinct events for each resource combinations. This results from an example run of $N = 2 \cdot 10^4$ IMGEP iterations with $k = 1, 2, 3$ and a warming set of $N_{init} = 1000$ iterations of random program synthesis, and 3 random mutations from the mutation operator.

- IMGEP succeeds in finding multiple interference cases for every resource combination.
- As opposed to other steady sampling distributions.

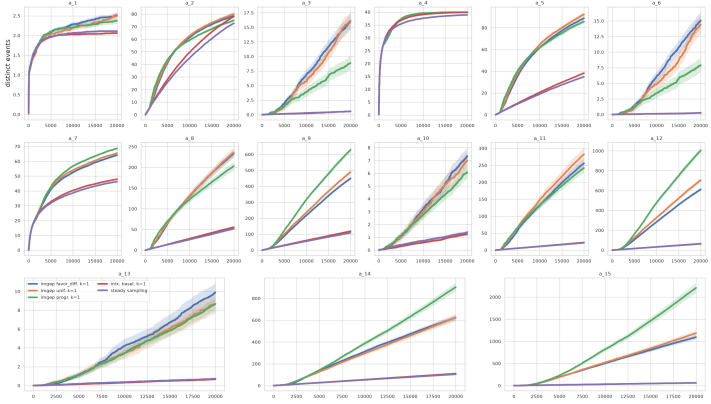


Figure – Evolution over the exploration of the number of distinct events for the different combinations of interference types. This results from the same example runs than presented on figure 5

Analysis of the dataset

We demonstrate a prospective analysis method for assessing the nature of the collected intereference events.

- What are most encountered features for an interference combination $a \in \mathcal{S}$?
- $\implies$ Analyze Distribution of vectors $\{r : z_r \in \mathcal{B}(r) | \forall r \in a\}$.
- One can end up with vectors containing too many features to be easily analyzed. $\implies$ One keeps only the most relevant features before observing their distribution.

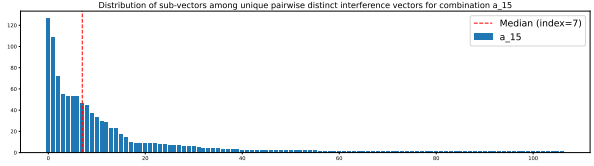


Figure – Among the set of pairwise distinct vectors for combination $a_{15} \in \mathcal{S}$, distribution of the subvectors featuring a selection of values. We see that median is 7, meaning that only 7 distinct vectors are responsible for 50 pourcents of the interference.

Analysis of the dataset

We demonstrate a prospective analysis method for assessing the nature of the collected intereference events.

- What are most encountered features for an interference combination $a \in \mathcal{S}$?
- $\implies$ Analyze Distribution of vectors $\{r : z_r \in \mathcal{B}(r) | \forall r \in a\}$.
- One can end up with vectors containing too many features to be easily analyzed. $\implies$ One keeps only the most relevant features before observing their distribution.

	DDR :bank	DDR sched. :bank	DDR sched. :addr	DDR sched. :bank attacker	L2 cache :evicted core id	L2 cache :causing core id
1	5	5	13	5	1	0
2	2	2	2	2	1	0
3	5	5	13	5	0	1
4	4	4	4	4	1	0
5	1	1	9	1	1	0
6	5	5	5	5	1	0
7	5	5	5	5	0	1

Table – Vectors of features inducing the 50 percent quantile of the interference for combination a_{15}. Only a few features from table are considered, as one wants to perform a tracktable analysis.

Analysis of the dataset

We demonstrate a prospective analysis method for assessing the nature of the collected intereference events.

- What are most encountered features for an interference combination $a \in \mathcal{S}$?
- $\implies$ Analyze Distribution of vectors $\{r : z_r \in \mathcal{B}(r) | \forall r \in a\}$.
- One can end up with vectors containing too many features to be easily analyzed. $\implies$ One keeps only the most relevant features before observing their distribution.

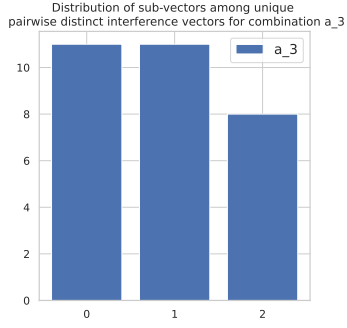


Figure – Among the set of pairwise distinct vectors for combination $a_3 \in \mathcal{S}$, distribution of the subvectors featuring a selection of values.

Analysis of the dataset

We demonstrate a prospective analysis method for assessing the nature of the collected intereference events.

- What are most encountered features for an interference combination $a \in \mathcal{S}$?
- $\implies$ Analyze Distribution of vectors $\{r : z_r \in \mathcal{B}(r) | \forall r \in a\}$.
- One can end up with vectors containing too many features to be easily analyzed. $\implies$ One keeps only the most relevant features before observing their distribution.

bank	addr	bank attacker	row attacker
0	16	1	5
4	4	0	3
4	4	1	1

Limitations

- The goal achievement strategy also consists in mutating programs based on the goal.
- The system could be more realistic, and a transition on a high fidelity simulator like gem5 could benefit the audit of this project.

Timing data

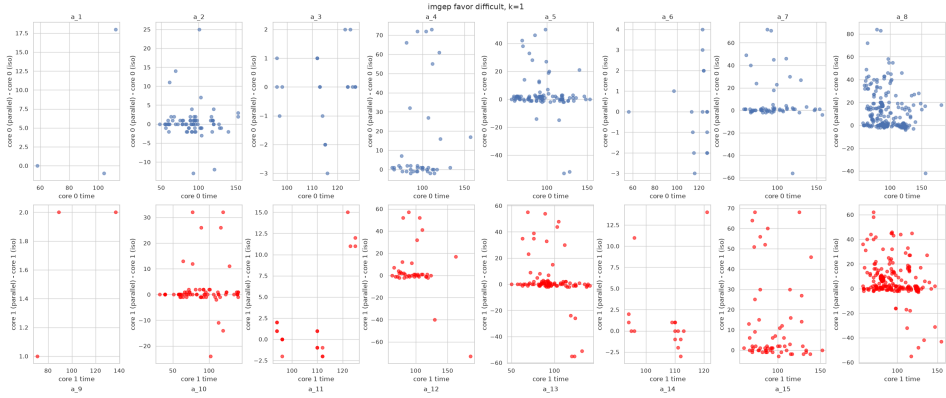


Figure – One observes that extending time does not determine the source of interference

Timing data

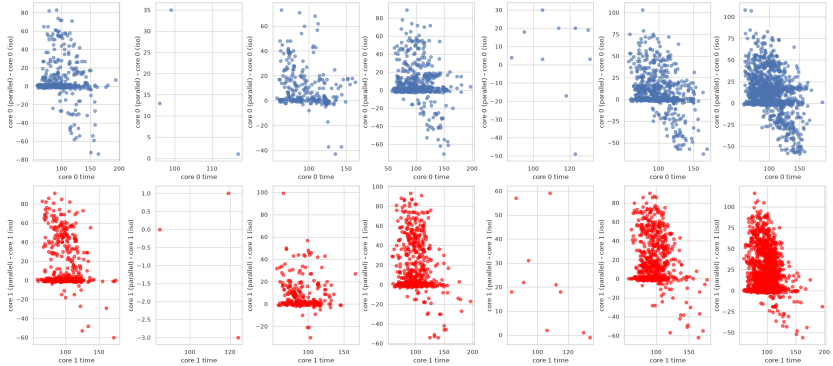


Figure – One observes that extending time does not determine the source of interference

Outline

1. Context

2. Our simplified simulator

3. Exploration method

4. Results

5. Conclusion

Conclusion

- The presented algorithm archiecture Intrinsically Goal Exploration Process relies on simple machine learning models to structure an autonomous agent that fixes his own goals, and on the establishment of a behavior and a parameter space
- Results demonstrate that automated discovery algorithms can successfully explore behaviors of a simplified simulated model of dual-core architecture, and encourage the idea of exploring more complex simulated platfoms reproducing real multicore architectures
- This approach encourages the idea synthesizing a tool to help researchers understand mechanisms that are sources of interference.